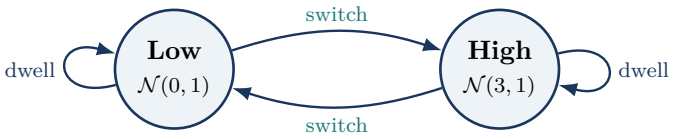
Introduction to Machine Learning — Spring 1405

Hidden Markov Anomaly Detection

From OC-SVM to Structured Sequence Models

Project report — theoretical and simulation questions



Student	Email
Mahyar MOHAMMADALIPOOR	mahyar.alipoor84@sharif.edu
Darya AZADDEL	darya.azaddel18@sharif.edu
Ali REZAEI	ali.rezaei.84@sharif.edu

What this report studies. We compare a static one-class detector (OC-SVM) with a sequence-aware model (HMAD). The dataset is built so that anomalies live in the *switching behaviour*, not in the raw values. If a method only looks at mean, variance, or range, it should be substantially weaker than a sequence-aware model.

Contents

1 Why this project matters **3**

2 Introduction to Anomaly Detection **4**

3 OC-SVM Baseline **5**

4 Latent Dependency and Real-World Motivation **8**

5 Hidden Markov Anomaly Detection **13**

6 Simplified HMAD Training Loop **16**

7 Full Comparison: OC-SVM vs. HMAD **18**

8 Conclusion **20**

1 Why this project matters

The main question of this project is simple, but it is easy to not notice how important it is: *does knowing the order of events really help us catch anomalies, or is a good static snapshot of the data (mean, variance, range) already enough?* In many anomaly detection courses, people only look at this static snapshot. This project asks us to compare the two ideas in a fair way. We build a dataset where the static snapshot is limited, and then measure how much a sequence-aware model does better.

Our dataset is a two-state regime-switching process. You can think of a machine that goes between a "Low" mode and a "High" mode. It stays in each mode for a random dwell time of 15–30 steps, and then it switches. Both states use the same emission distributions in every sequence that we generate ($\mathcal{N}(0,1)$ for Low, $\mathcal{N}(3,1)$ for High). The anomalies never change these distributions. They only change the *switching behaviour*:

- **Chatter anomaly**: the system starts to jump between states every 1–3 steps, instead of staying for 15–30 steps.
- **Stuck anomaly**: the system stays in one state for 55–75 steps, which is much longer than a normal dwell time.

No single reading is out of range in these two cases. So if a detector wants to find them, it has to look at *timing*, not at how large the values are. This is exactly the difference between OC-SVM and HMAD that we study in the rest of this report.

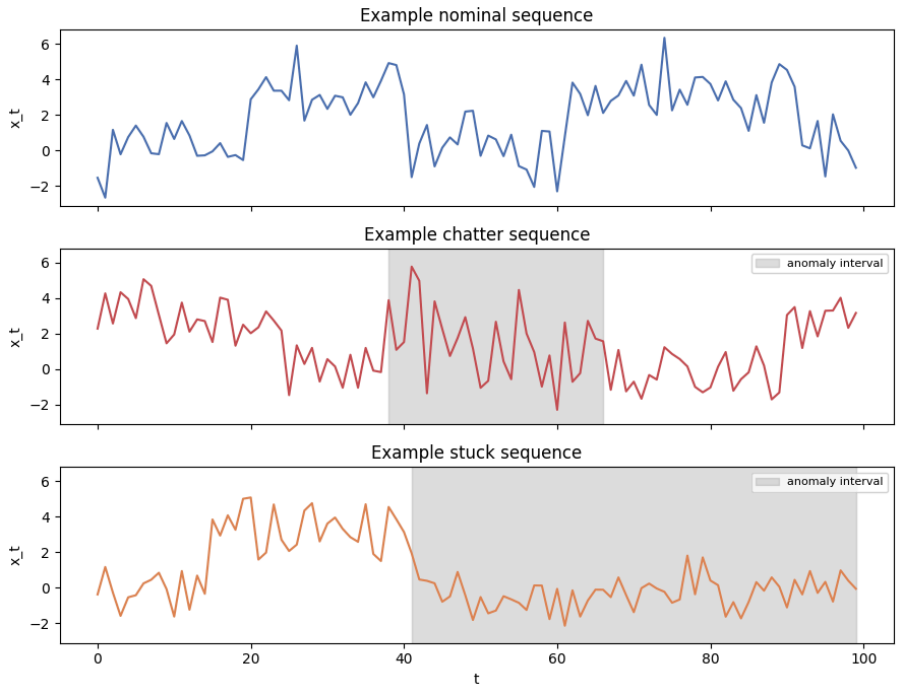


Figure 1: One example sequence of each type. The values stay in the same two bands in all three plots. What changes is how long the signal stays in a band before it switches.

Important detail: the main dataset is a *semi-Markov / regime-switching* process with random dwell times, not a plain first-order HMM with a fixed transition matrix. Later, HMAD still fits an ordinary HMM transition matrix A; we interpret the learned A as an *effective one-step* summary of the dwell behaviour, not as the literal generator.

Dataset facts used in the experiments

- Source: self-generated synthetic sequences (seed 1405).
- One sequence = one length-$T = 100$ trajectory of scalar observations.
- Training: 200 nominal sequences (20,000 timesteps), no anomalies.
- Main held-out test: 150 nominal + 50 anomalous (chatter/stuck mixed).
- Emissions: Low $\mathcal{N}(0, 1)$, High $\mathcal{N}(3, 1)$; nominal dwell 15–30; chatter dwell 1–3 inside a random window; stuck dwell 55–75.
- Anomaly intervals are stored with each sequence and highlighted in the plots.

Compliance with the eight dataset requirements

1. Order matters: anomalies change switching/dwell, not the emission families.
2. Train is all-nominal; test has sequence-level labels.
3. ≥ 100 nominal train sequences and ≥ 5000 timesteps (we use 200×100).
4. Test has ≥ 20 anomalies; contamination sweeps keep anomaly count fixed at 20.
5. Fixed length $T = 100$ for all sequences (avoids padding/length confound).
6. Sequences are independent draws, not slices of one long signal.
7. Anomaly type/position are random; no post-hoc cherry-picking.
8. Not a trivial magnitude anomaly: per-sequence means overlap strongly; Mann-Whitney tests give p=0.29 (chatter) and p=0.66 (stuck). Other summaries retain partial information, especially global standard deviation for stuck; static OC-SVM remains above chance. Its AUC is about 0.72-0.80, so we only claim that the main anomaly signal is structural.

A quick check pools 150 nominal / 76 chatter / 74 stuck sequences. Group means of the raw-value means stay close (about 1.51, 1.55, 1.46), but for stuck sequences global standard deviation retains partial information. The anomaly definition itself remains structural.

2 Introduction to Anomaly Detection

Theoretical Question 1.

Give one example of a point anomaly and one example of a sequence anomaly. Explain why the sequence anomaly may not be detectable by looking at each observation independently.

Answer. A point anomaly is one reading that looks wrong by itself — for example a vibration spike on a turbine bearing that is far outside the normal range. A sequence anomaly is different. Think about the path 60°C → 85°C → 60°C in a few seconds. None of these temperatures is strange alone, but heating and cooling that fast may be impossible for the machine. A detector that scores each reading one by one sees "85°C, this is not that rare" and then it continues. It never asks if the *transition* is possible. This is exactly what our chatter and stuck anomalies do.

Every single value is a normal sample from $\mathcal{N}(0,1)$ or $\mathcal{N}(3,1)$. So a detector that only looks at values one at a time cannot see the problem.

Theoretical Question 2.

Why is anomaly detection harder than ordinary binary classification when labeled anomalous examples are unavailable or unreliable?

Answer. In normal binary classification, we see labelled examples from both classes, so we can learn a boundary between them. Anomaly detection does not give us this. Anomalies are rare, they can have many different types, and we often do not have good training labels for them. The model can only learn what "normal" looks like, and then it treats a large deviation as suspicious. This is a one-sided estimation problem, and the model also has to work for anomaly types that it has never seen. This is harder than fitting a boundary between two known classes.

Theoretical Question 3.

Suppose $\nu = 0.05$. What does this say about the amount of probability mass that the ideal detector is allowed to place outside the nominal region?

Answer. Formally, the ideal level set satisfies $P(X \notin L_\nu) = \nu$. So $\nu = 0.05$ means about 5% of the true nominal probability mass is allowed outside the nominal region. In other words, if we sample many points from the real "normal" process, about 1 in 20 points can fall outside the learned boundary only because of natural variation, not because something is really wrong.

3 OC-SVM Baseline

Theoretical Question 4.

Why is OC-SVM called a "one-class" method? Explain how its training setting differs from ordinary binary classification.

Answer. OC-SVM is called "one-class" because we train it on data from only one class — the nominal class — and we do not use labelled anomalies. It does not try to separate two classes. It finds a region that contains most of the nominal training points, and it treats the outside as the unseen anomaly class. Normal binary classification needs labelled examples from both classes to build its boundary. OC-SVM never gets this second class.

Theoretical Question 5.

Why is OC-SVM a meaningful baseline for this project, even though the project later studies sequence-aware anomaly detection?

Answer. OC-SVM is a standard kernel method, and people understand it well. So it is a fair baseline for "how far can we go with a fixed-vector one-class detector that does not explicitly model transitions?" Flattened features still keep absolute positions, so order is not fully deleted; what is missing is an explicit transition/dwell representation. HMAD adds that structure. On our dataset OC-SVM stays in the roughly 0.72–0.80 AUC range, while HMAD is much higher (Section 6).

Theoretical Question 6.

What is the geometric meaning of separating the mapped training data from the origin in the OC-SVM feature space?

Answer. OC-SVM looks for a hyperplane $\langle w, \phi(x) \rangle = \rho$ such that as much of the mapped nominal data as possible sits on the side $\langle w, \phi(x) \rangle \geq \rho$, away from the origin. The origin is used as the default "anomalous" reference point. In geometry, this makes a half-space (or, with the kernel, a curved region in the original input space). This region contains most of the nominal data, as far from the origin as the margin $\rho/\|w\|$ allows.

Theoretical Question 7.

Why are slack variables ξ_i needed in the OC-SVM objective? Relate your answer to noisy or contaminated training data.

Answer. If we do not use slack variables, every training point would have to satisfy $\langle w, \phi(x_i) \rangle \geq \rho$ exactly. Then the boundary would have to fit even the most extreme training example, which can be noisy or even mislabeled. Slack variables ξ_i let a controlled part of the points break the margin, but they pay a cost for this. So a few dirty or outlying training samples do not change the whole decision region.

Theoretical Question 8.

Explain the role of ν in OC-SVM. What may happen if ν is chosen much smaller or much larger than the true contamination level?

Answer. ν controls how much of the training data the model is ready to reject. If we set it much smaller than the true contamination level, the model becomes too soft. It puts a boundary around noise that it should have rejected, and then anomalies mix with the accepted region. If we set it much larger, the model becomes too strict. It rejects a part of truly normal data and the false positive rate goes up. In practice we try a few values and we check which one gives a better balance between precision and recall. Section 6.3 shows that this sweep has a real effect when the dataset is not too easy to separate.

Theoretical Question 9.

Why does converting a sequence into a fixed-length vector not necessarily mean that temporal dependency has been properly modeled?

Answer. If we flatten a sequence, or if we summarize it into a fixed-length vector, we still may fail to model *conditional* structure between time steps, unless the features are designed for that. Flattening keeps absolute positions, and an RBF kernel can use interactions across coordinates; it does *not* treat dimensions as independent. But neither flattening nor simple summaries explicitly count transitions or dwell times. A window mean, for example, tells the average level in a chunk of time, not how fast the signal switched inside that chunk. Two sequences with very different switching behaviour can still look similar after a weak summary map.

Simulation Question 1.

Implement an OC-SVM baseline using scikit-learn on the selected dataset. Report the feature representation, preprocessing, kernel, ν, kernel parameters, and ROC-AUC.

Answer. We built the summary-statistics baseline ([mean, std, min, max, median]) with a StandardScaler that we fit only on the training data, an RBF kernel, $\nu = 0.1$, and `gamma="scale"`. On the held-out test set (150 nominal + 50 anomalous), it reached **ROC-AUC** $= 0.7208$. This is clearly better than chance, but it is not close to perfect. This is not surprising, because global summaries do not explicitly encode the switching/dwell dynamics of our anomalies.

Simulation Question 2.

Compare at least two sequence feature representations for OC-SVM, such as flattened features and summary or window-based features. Which representation works better, and why?

Answer. We compared three representations. We used the same preprocessing and $\nu = 0.1$ for all of them:

Representation	ROC-AUC
Flattened ($\phi(x) = x \in \mathbb{R}^{100}$)	**0.7963**
Global summary (5-d)	0.7208
Window-based (10 windows, 20-d)	0.6873

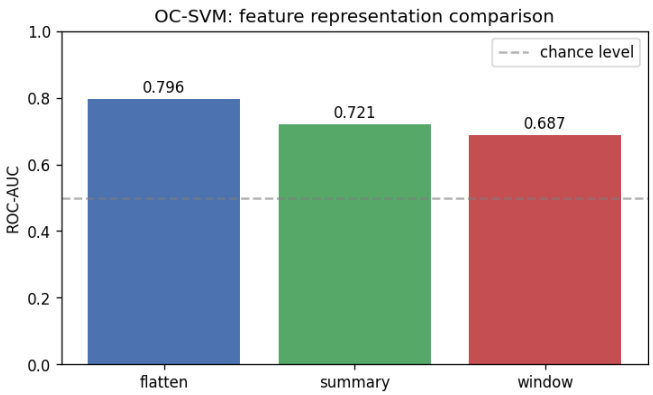


Figure 2: Representation comparison for the OC-SVM baseline.

Flattening is the best one here, and the reason is quite intuitive. Every sequence has the same length, and the flattened vector keeps the position of each value. So the RBF kernel can notice some rough patterns in *where* the switches often happen, and also how variable each region is. It does this only as a side effect of comparing raw vectors point by point. Still, none of the three representations really counts transitions. So all three of them miss some real signal. This is the gap that HMAD closes in Section 6.

For later experiments, flattened is pre-specified; this test-set comparison is descriptive only.

Simulation Question 3.

Evaluate OC-SVM for several values of ν, for example $\{0.01, 0.05, 0.1, 0.2\}$. Report how ROC-AUC changes and briefly interpret the result.

Answer. We swept $\nu \in \{0.01, 0.05, 0.1, 0.2\}$ on the flattened representation. The AUC values were 0.7957, 0.7961, 0.7963, 0.7988. The curve is almost flat here, because the hard part of this dataset is hard because features capture structure indirectly. The numbers still move a little: nu still affects the boundary, although the AUC effect is small; this is not a saturated problem.

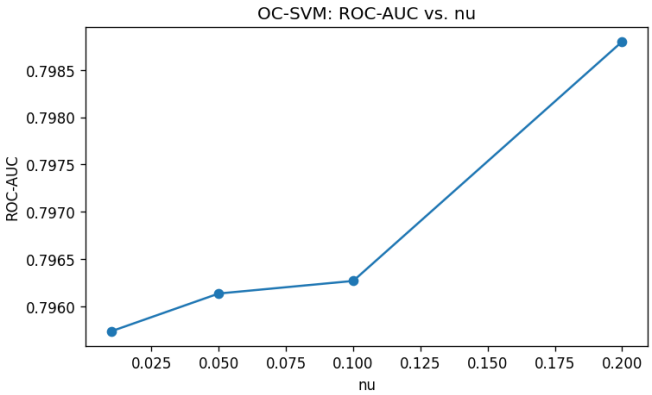


Figure 3: ROC-AUC as a function of ν, flattened representation.

Baseline limitation and motivation for HMAD. All three OC-SVM variants stay well below a perfect score (best about 0.80). The shared reason is structural: none of them has the idea of a hidden state or a transition count. They can only notice switching in an indirect way, for example through variance or position features. A model that decodes hidden states and counts transitions should do much better. This is the main idea of the next two sections.

4 Latent Dependency and Real-World Motivation

Theoretical Question 10.

Explain why a detector that only looks at individual observations may fail to detect a sequence anomaly. Give one example where each individual observation is normal, but the ordering is suspicious.

Answer. A detector that scores single observations only asks "is x_t possible on its own?" It has no way to punish a strange move from x_{t-1} to x_t. Example: `Port Scan` → `Login` → `Data Download`. Each event type is common by itself, so a marginal model gives a high score to each one. But this specific order is a known attack pattern, and a transition-aware model would flag it quickly.

Theoretical Question 11.

For the server-log example above, explain why the sequence `Port Scan` → `Login` → `Data Download` could have low probability under a nominal transition model, even if each event has high marginal probability.

Answer. Under a Markov model, the probability of a sequence is a *product* of conditional transition probabilities: $P(\text{seq}) = \pi(\text{Scan}) \cdot A(\text{Scan} \to \text{Login}) \cdot A(\text{Login} \to \text{Download})$. Even if every event is common alone, one specific conditional — for example $A(\text{Scan} \to \text{Login})$ — can be small under normal system behaviour. Then one small factor is enough to make the whole product small.

Theoretical Question 12.

Consider a two-state Markov chain with states 1 = normal and 2 = high-load. What does a large value of A_{11} mean? What does a large value of A_{12} mean?

Answer. A large A_{11} means that, in one step, the chain is likely to *stay* in the normal state. A large A_{12} means that a jump from normal to high-load is likely. These are one-step transition probabilities. A fixed large A_{ii} makes long stays more probable, but it does *not* define a uniform dwell-time distribution: under a first-order Markov chain the dwell length is geometric. Our main dataset is semi-Markov (explicit random dwell ranges), so the learned HMAD matrix $\hat{A}$ is only an effective one-step summary of that dwell behaviour. Nominal dwells are about 15–30 steps (diagonal near 0.95–0.96). Chatter shortens dwells inside a window; stuck forces one very long dwell.

Theoretical Question 13.

What is the difference between a Markov chain and a hidden Markov model? In your answer, distinguish between hidden states z_t and observations x_t.

Answer. In a Markov chain, the states z_t *are* the observed data. In an HMM, the states z_t are latent. We never see them directly. We only see emissions x_t that come from the state that the system is in. The chain still controls how z_t changes. The emission model controls how x_t is produced given z_t. So inference in an HMM has two steps: first we infer the hidden path, and then we interpret the observations through this path.

Theoretical Question 14.

In the Gaussian-emission HMM used in this project, each hidden state i has parameters μ_i and σ_i^2. What do these parameters represent?

Answer. Each hidden state i has a mean μ_i and a variance σ_i^2. They describe the Gaussian that generates observations while the system is in that state. In our dataset these match Low ($\mu \approx 0,\ \sigma^2 \approx 1$) and High ($\mu \approx 3,\ \sigma^2 \approx 1$). Section 6 shows that the training loop finds these values almost exactly, only from the data.

Simulation Question 4.

Write a Python function that computes the log-probability of a state sequence $z = (z_1, \ldots, z_T)$ under a given initial distribution π and transition matrix A: $\log P(z \mid \pi, A) = \log \pi_{z_1} + \sum_{t=2}^{T} \log A_{z_{t-1}, z_t}$. Use log-probabilities rather than raw products to avoid numerical underflow.

Answer. We implemented the log-probability of a state sequence as $\log P(z \mid \pi, A) = \log \pi_{z_1} + \sum_{t=2}^{T} \log A_{z_{t-1}, z_t}$. We checked it against the direct product for $\pi = (0.6, 0.4)$, $A = \left(\begin{smallmatrix} 0.9 & 0.1 \\ 0.2 & 0.8 \end{smallmatrix}\right)$ and $z = (0, 0, 0, 1, 1, 0)$. The log form gives $\log P(z) = -4.857$, so $P(z) = 0.007776$. This matches the direct product 0.007776 exactly.

Theoretical Question 15.

Why is brute-force decoding of the most likely hidden state sequence infeasible for long sequences? What is the number of possible paths when there are K states and sequence length T?

Answer. If we try to list every possible hidden path, this is not possible in practice, because there are K^T of them. Even for small numbers like $K = 2$ and $T = 100$, we get 2^{100}, which is much larger than anything we can check one by one. Dynamic programming (Viterbi) avoids this. It only keeps the best partial path that ends in each state at each time step. Then the cost becomes $O(K^2 T)$.

Simulation Question 5.

Implement a function that generates a sequence from a two-state Gaussian-emission HMM. Your function should take π, A, emission means (μ_1, μ_2), emission variances (σ_1^2, σ_2^2), and sequence length T. It should return both the hidden state sequence z and the observed sequence x.

Answer. We generated a 100-step sequence from a two-state Gaussian HMM. We used the same emission parameters as our real dataset ($\mu = (0, 3)$, $\sigma^2 = (1, 1)$, $A_{11} = A_{22} = 0.955$).

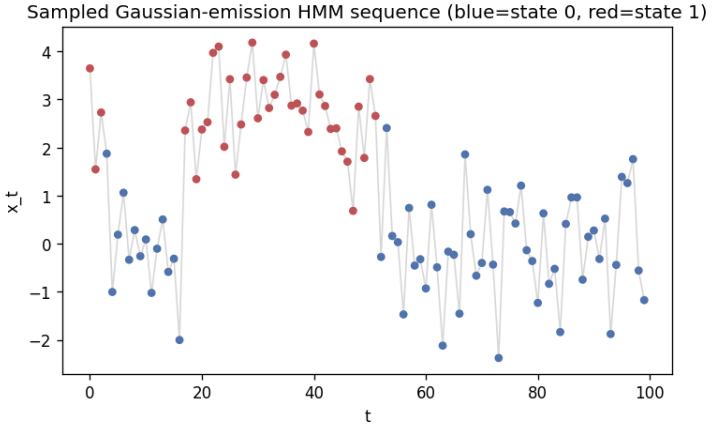


Figure 4: A sampled HMM sequence, coloured by its true hidden state.

Simulation Question 6.

Implement log-space Viterbi decoding for a two-state Gaussian-emission HMM. Test it on a sequence generated by your HMM generator. Report the fraction of time steps where the decoded hidden state matches the true generated hidden state.

Answer. Then we ran log-space Viterbi decoding on this sequence.

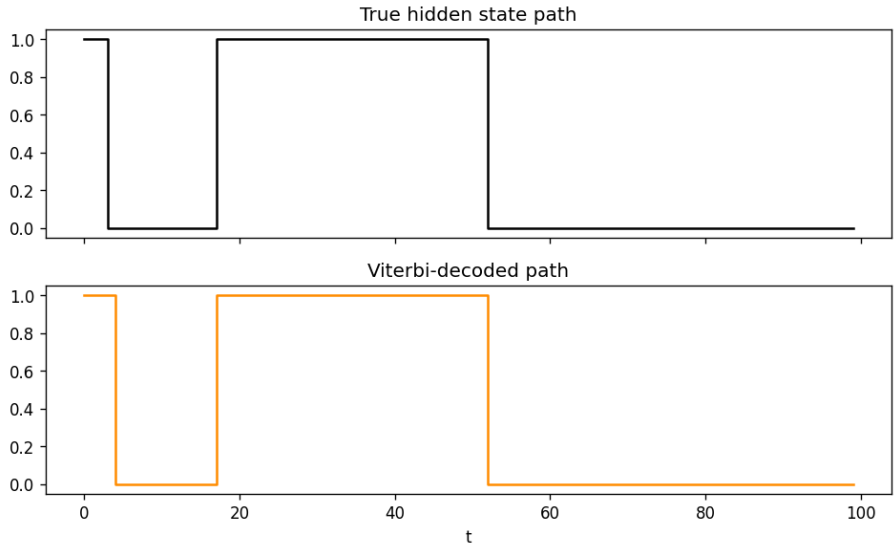


Figure 5: True hidden path vs. Viterbi-decoded path on the same sequence.

The decoding accuracy against the true hidden states was **99%** on this run. The emission means are well separated (0 vs. 3, both with variance 1) and the transitions are persistent, so the Gaussians almost do not overlap in the tails that matter for state assignment. Because of this, Viterbi recovers the path almost perfectly. This will not be true for every random seed, but it shows that the algorithm works correctly before we use it on more messy anomalous sequences.

Simulation Question 7.

Compare ordinary probability multiplication and log-space computation for a long Markov chain sequence. Generate a sequence of length $T = 5000$, compute its probability both as a raw product and as a log-probability sum, and show when the raw product begins to suffer from numerical underflow.

Answer. For a length-5000 chain, the raw product goes to exactly 0.0 in double precision at prefix length ≈ 1990. The log-probability sum stays finite all the way.

This is not only a small numerical detail. After $t \approx 2000$, the raw product cannot compare any two hypotheses anymore, because both of them become zero. Any algorithm (also Viterbi) that compares products of probabilities on sequences longer than a few hundred steps *must* work in log-space. If it does not, it fails in a silent way.

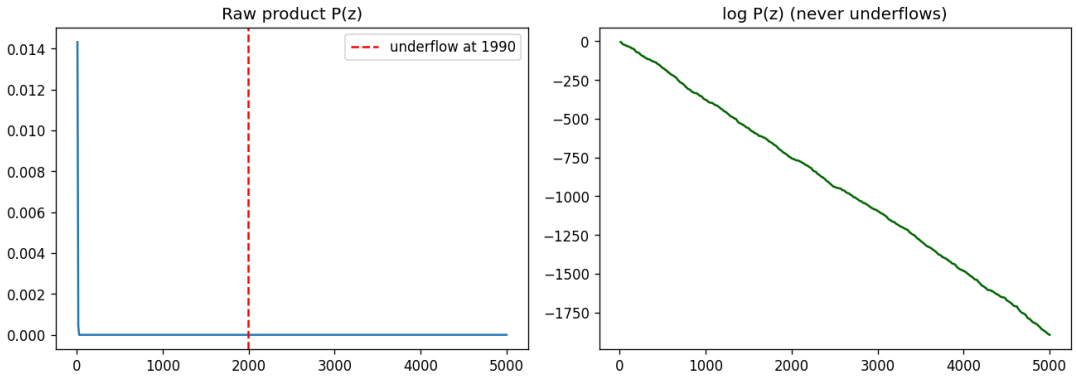


Figure 6: The raw product goes to zero. The log-sum never does.

Theoretical Question 16.

In one or two paragraphs, explain why Viterbi decoding is useful for HMAD. What role does the decoded hidden state sequence play in constructing the feature vector $\Psi(x, z)$?

Answer. Viterbi decoding is what turns an observed sequence into something that HMAD can use to build features. HMAD does not score raw observations directly. It scores a joint feature vector $\Psi(x, z)$ that depends on *both* the values and a hidden-state explanation of how they were created. Since z is never given, something has to produce it. It also has to be the most possible path under the current model, not a random guess. If we use a random guess, $\Psi(x, z)$ would describe a story that the data did not really tell. Viterbi gives exactly this: the hidden path that best matches the observations with the learned transition and emission structure. After this path is fixed, $\Psi(x, z)$ can count how often each transition happened and what was emitted in each state. This is the structural fingerprint that OC-SVM's fixed feature map cannot produce. In short, Viterbi is the step that makes the "hidden" part of Hidden Markov Anomaly Detection concrete enough, so we can compute a feature vector from it.

Bonus Theoretical Questions

Theoretical Question 17.

A stationary distribution v of a Markov chain satisfies $v^\top A = v^\top$ and $\sum_i v_i = 1$. For the transition matrix $A = \left(\begin{smallmatrix} 0.95 & 0.05 \\ 0.10 & 0.90 \end{smallmatrix}\right)$, compute the stationary distribution and interpret it as the long-run fraction of time spent in each state.

Answer. For $A = \left(\begin{smallmatrix} 0.95 & 0.05 \\ 0.10 & 0.90 \end{smallmatrix}\right)$, if we solve $v^\top A = v^\top$, we get $v = (2/3,\ 1/3)$ (we also checked this numerically with the eigenvector of $A^\top$ for eigenvalue 1). In the long run the system spends two times more time in state 1 than in state 2. This happens because state 1 is "stickier" compared to how often we enter it from state 2.

Theoretical Question 18.

The entropy rate of a Markov chain with stationary distribution v and transition matrix A is $H = -\sum_i v_i \sum_j A_{ij} \log A_{ij}$. Explain qualitatively how a transition matrix with highly predictable transitions differs from a transition matrix with nearly uniform transitions. How might this be useful for anomaly detection?

Answer. A transition matrix with very predictable transitions (rows close to one-hot) has a low entropy rate. A matrix with almost uniform rows has a high entropy rate. Low entropy means that the next move of the system is easy to guess if we know the current state. This is what we expect from nominal "dwell and switch" dynamics. A sudden rise in local entropy (more unpredictable switching) can itself be used as an anomaly signal, even if the emission values do not change. This is a more formal way to describe what our chatter anomaly does to the transition structure.

Theoretical Question 19.

For an HMM, the marginal likelihood of an observation sequence is $P(x \mid \lambda) = \sum_z P(x, z \mid \lambda)$. Explain why computing this sum by brute force is infeasible. Then state the main idea of the Forward algorithm and how it reduces the computational complexity.

Answer. If we compute $P(x \mid \lambda) = \sum_z P(x, z \mid \lambda)$ by brute force, we have to sum over K^T paths. This is not possible for any serious T. The Forward algorithm solves this in the same way that Viterbi solves the max problem. It recursively computes $\alpha_t(j) = P(x_1, \ldots, x_t, z_t = j)$ by summing (not maximizing) over the previous state. Then the cost becomes $O(K^2 T)$.

Theoretical Question 20.

Posterior decoding chooses the most likely state at each individual time step: $\hat{z}_t = \arg\max_i P(z_t = i \mid x, \lambda)$. Viterbi decoding chooses the globally most likely state sequence. Explain why posterior decoding can sometimes produce an invalid or unlikely global path, while Viterbi respects the transition structure of the model.

Answer. Posterior decoding chooses $\hat{z}_t = \arg\max_i P(z_t = i \mid x, \lambda)$ independently at every time step. Because each t is optimized alone, the final path can contain a transition that the model almost never allows (for example jumping between states that the transition matrix says almost never happen one after another). Viterbi instead optimizes the whole path together, so every transition that it reports is one that the model really sees as possible.

5 Hidden Markov Anomaly Detection

Theoretical Question 21.

OC-SVM uses a fixed feature map $\phi(x)$, while HMAD uses a joint feature map $\Psi(x, z)$ with a maximization over hidden states. Explain in your own words what this maximization is doing. Why can it help identify anomalies that a fixed feature map would miss? Use an example to make your case.

Answer. In the formal HMAD score, $f(x) = \max_z \langle w, \Psi(x, z) \rangle - \rho$ searches over hidden-state

explanations of x and uses the best one. If even the most generous explanation looks abnormal, the score stays low. In our *simplified course implementation*, we do not optimize over all z at scoring time: we first decode one path z^* by Viterbi under the learned HMM, then score $\Psi(x, z^*)$. A concrete example: a "stuck" sequence needs one dwell of 55–75 steps. Ordinary Such a long dwell is atypical under the learned transition structure and yields unusual transition features, even though the observations remain normal emissions.

Theoretical Question 22.

Suppose two sequences $x^{(1)}$ and $x^{(2)}$ have identical summary statistics (same mean, variance, min, and max). Can HMAD still assign them different anomaly scores? Explain why or why not.

Answer. Yes. $\Psi(x, z)$ depends on the Viterbi-inferred path, not on marginal statistics of x. This is not only a theory idea in our project. Section 0.5 shows strong overlap in per-sequence means; stuck sequences retain some variance information. But the full HMAD pipeline separates them with AUC ≈ 0.999, while a summary-statistic OC-SVM only gets AUC ≈ 0.72. Same values, different structure, and very different scores.

Theoretical Question 23.

For $K = 2$ hidden states and scalar observations ($d = 1$), write out all entries of $\Psi(x, z)$ explicitly. How many entries does the feature vector have? What does each entry represent?

Answer. For $K = 2$, $d = 1$ the theoretical joint feature has $K^2 = 4$ transition counts plus $K \cdot d = 2$ emission aggregates, so six entries. In the handout theory these can be raw counts / sums; in our implementation we use the normalized version $\Psi(x, z) = [c_{00}, c_{01}, c_{10}, c_{11}, e_0, e_1]$, where c_{ij} are transition counts divided by $T - 1$ and e_i is the mean observation in state i (with a small ε if a state is empty).

Simulation Question 8.

To build intuition for why the joint feature map captures structure that summary statistics miss, construct a toy experiment in Python.

- Generate two sequences of length $T = 100$ with the same 50 values from $\mathcal{N}(0, 1)$ and 50 values from $\mathcal{N}(4, 1)$, but arranged differently: contiguous blocks vs. a random shuffle.
- Verify that the two sequences have nearly identical global summary statistics, and plot both sequences.
- Assign hidden states with a simple threshold, compute $\Psi(x, z)$ for both sequences, and compare which entries differ the most.

Answer. We used the exact handout construction: the same 100 values in both sequences. Global summary statistics match exactly (mean/std/min/max all equal). After a threshold state assignment ($z_t = 1$ if $x_t \geq 2$), the joint features differ mainly in the transition block: absolute differences about 0.20–0.21 on $c_{00}, c_{01}, c_{10}, c_{11}$, while emission means e_0, e_1 stay the same. So Ψ sees order even when summaries do not.

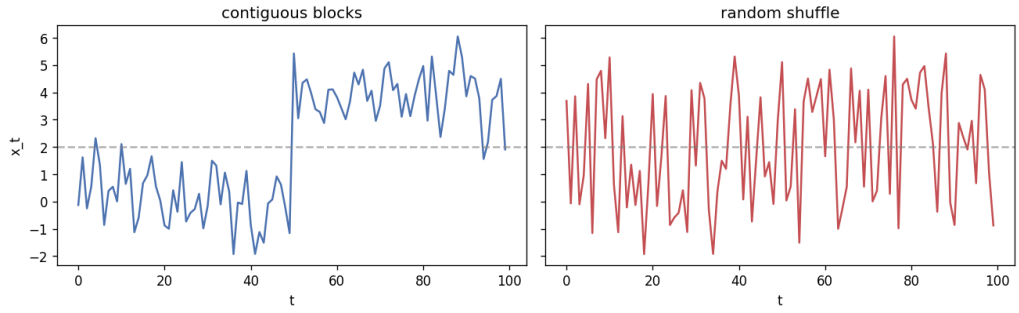


Figure 7: SQ8 toy sequences: identical values, contiguous blocks vs. random shuffle.

Simulation Question 9.

Write a Python function `compute_joint_features(x, z, K)` that computes $\Psi(x, z)$ for a scalar sequence x and a hard state assignment z, using $\phi(x_t) = x_t$.

- Return a vector of length $K^2 + K$: normalized transition counts, then per-state mean emissions.
- Compute $\Psi(x_i, z_i)$ for a larger batch of nominal and anomalous sequences.
- Plot the feature matrix as a heatmap. Which entries of Ψ visually separate the two groups?

Answer. Independently of the main regime-switching dataset, we generated the handout toy batch: 200 nominal sequences from $\mathcal{N}(0, 1)$ and 50 anomalous sequences with a random 20-step block from $\mathcal{N}(4, 1)$. The heatmap separates the groups mainly through c_{00}, c_{11} and especially e_1 (mean emission in the high state), which jumps from about 2.16 on nominal sequences to about 3.94 on anomalous ones.

Theoretical Question 24.

The training algorithm alternates between a Viterbi step and an OC-SVM step. What would happen if the algorithm were run for only one iteration (i.e. $M = 1$)? How does running more iterations potentially improve the learned model?

Answer. With $M = 1$, hidden paths are decoded only once from the initial (K-means) HMM parameters. In our implementation the OC-SVM does *not* feed back into the HMM update: each iteration does Viterbi $\rightarrow$ update (μ, σ^2, A) from the decoded paths $\rightarrow$ train OC-SVM on $\Psi(x, z)$. More iterations still help because better HMM parameters give better Viterbi paths, which give better Ψ features for the one-class step. By iteration 2 only 0.56% of state assignments still change, and the learned means are $(-0.004, 3.011)$ against true $(0, 3)$.

Theoretical Question 25.

During prediction, why is it necessary to run Viterbi decoding on the test sequence rather than using, for example, a fixed or randomly assigned hidden state sequence? What would go wrong if you skipped the Viterbi step and simply used $z = (1, 1, \ldots, 1)$ for every test sequence?

Answer. A fixed $z = (1, \ldots, 1)$ throws away all transition information. It also reduces every sequence's emission features to almost only its mean. In a concrete way: with a fixed path,

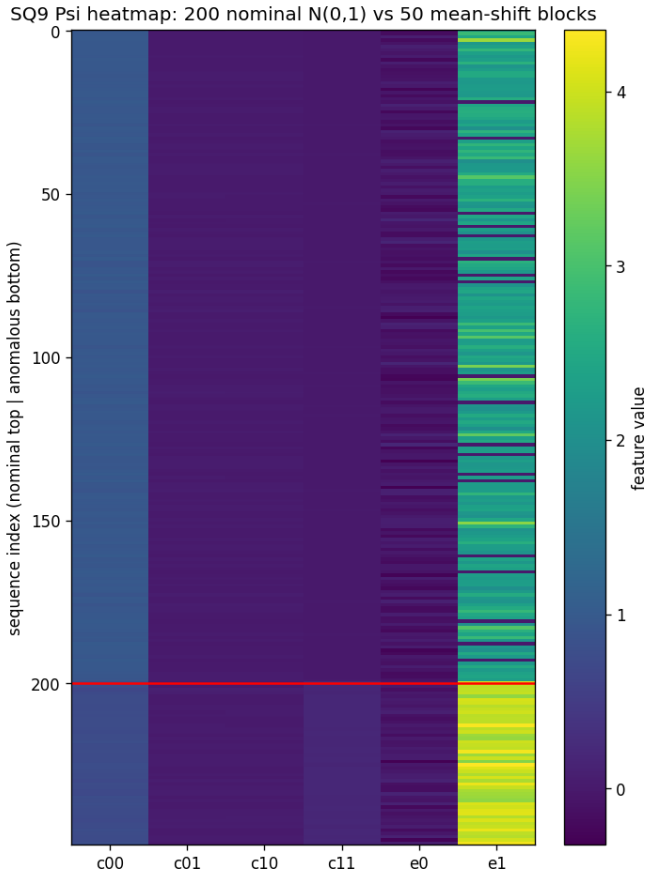


Figure 8: SQ9 $\Psi(x, z)$ heatmap: 200 nominal (top) vs. 50 mean-shift-block anomalies (bottom).

$c_{11} = 1$ for every sequence, no matter if it was chatter, stuck, or nominal. So the model cannot separate them on the transition features anymore. Section 6 shows that this is not only a theoretical worry. If we remove Viterbi, HMAD's AUC drops from 0.9988 all the way to 0.50, which is chance level.

6 Simplified HMAD Training Loop

Simulation Question 10.

Implement the simplified HMAD training loop (Algorithm 1) in Python with $K = 2$ hidden states and Gaussian emissions.

- Initialize emission parameters using K-means on pooled observations. Set $A_{kk} = 0.9$.
- Run up to $M = 10$ iterations. After each iteration, record the fraction of changed hidden-state assignments and the mean OC-SVM decision score.
- Plot both quantities vs. iteration. When do the hidden states stabilize?
- Print the learned emission means and transition matrix. Do they match the data-generating process?

Answer. We initialized the emission parameters with K-means ($K = 2$) on all pooled observations. We set the initial transition matrix to $A_{kk} = 0.9$, and we ran the alternating Viterbi/OC-SVM loop for up to 10 iterations.

The fraction of changed assignments drops from 1.0 (first Viterbi pass) to 0.0056 by iteration

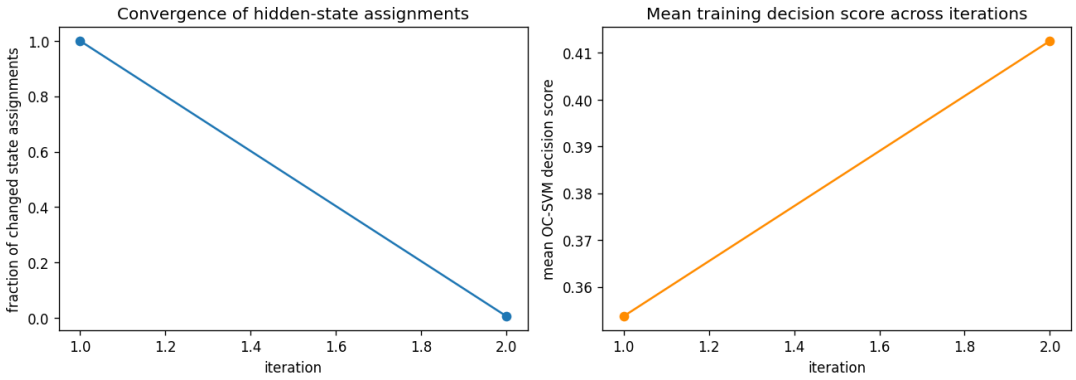


Figure 9: Convergence of hidden-state assignments and the mean OC-SVM decision score across iterations.

2, and after that it stays almost flat. The loop converges almost immediately here, because the two emission clusters are well separated, and K-means initialization is already close to the right answer. At convergence:

$$\hat{\mu} = (-0.004,\ 3.011), \qquad \hat{\sigma}^2 = (0.985,\ 0.986), \qquad \hat{A} = \begin{pmatrix} 0.959 & 0.041 \\ 0.041 & 0.959 \end{pmatrix}.$$

These are close to the effective dwell behaviour of the semi-Markov generator (means near $(0, 3)$, diagonal near 0.96). We treat $\hat{A}$ as an empirical one-step HMM approximation, not as a claim that the generator itself was a fixed first-order Markov chain.

HMAD training / prediction pseudocode

```
# Training
init (mu,s2,A,pi) by K-means + A_kk=0.9
for m = 1..10:
    z_i = Viterbi(x_i; pi,A,mu,s2) for all train i
    update (mu,s2,A) from {z_i}
    Psi_i = joint_features(x_i,z_i)
    fit StandardScaler + OC-SVM on {Psi_i}
    stop if <1% state assignments changed
final consistency: Viterbi + Psi + refit scaler/OC-SVM under final HMM

# Prediction
z* = Viterbi(x; final HMM)
score = OC-SVM(decision_function(scale(Psi(x,z*))))
anomaly_score = -score
```

7 Full Comparison: OC-SVM vs. HMAD

Simulation Question 11.

Using the trained HMAD model from the previous simulation question, implement the full prediction pipeline and evaluate it on the held-out test set.

- For each test sequence x, run Viterbi to obtain z^*, compute $\Psi(x, z^*)$, and evaluate the OC-SVM score $f(x)$.
- Report ROC-AUC and compare it to the OC-SVM baseline from Section 3.
- Repeat the evaluation with a trivial state assignment $z_t = 1$ for all t (skip Viterbi). How does the AUC change?
- Plot the Viterbi-decoded hidden path for one correctly classified normal sequence and one correctly classified anomalous sequence.

Answer. We used the trained HMAD pipeline (Viterbi decode → compute $\Psi(x, z^*)$ → scaled OC-SVM score) on the same held-out test set as the baseline:

Method	ROC-AUC
OC-SVM baseline (flattened features)	0.7963
HMAD (Viterbi-based)	**0.9988**
restricted to nominal + chatter	0.9994
restricted to nominal + stuck	0.9983
HMAD without Viterbi (fixed $z = 0$)	0.5000

The most important number in the report. If we remove hidden-state inference, HMAD falls to chance level. Sequence-awareness is not a small extra trick here — it is the whole mechanism that does the work.

The Viterbi ablation is the most telling number in the whole report: if we take out hidden-state inference, HMAD falls straight to chance level, because $\Psi(x, z)$ becomes a constant transition block plus a single-state emission mean. Sequence-awareness is not a small bonus here — it is the whole mechanism that does the work.

The decoded paths make the mechanism easy to see. The nominal example shows long clean dwells. The chatter/stuck examples show fast switching or one long dwell inside the marked anomaly interval. This is the structural signature HMAD's transition features are built to detect.

Method	Precision	Recall	F1
OC-SVM baseline	0.363	0.820	0.503
HMAD	0.704	1.000	0.826

Contamination sensitivity

We keep the handout fractions $r_{\text{test}} \in \{0.01, 0.05, 0.10, 0.20\}$ and fix the anomaly count at 20 (the required minimum), increasing the nominal pool as needed:

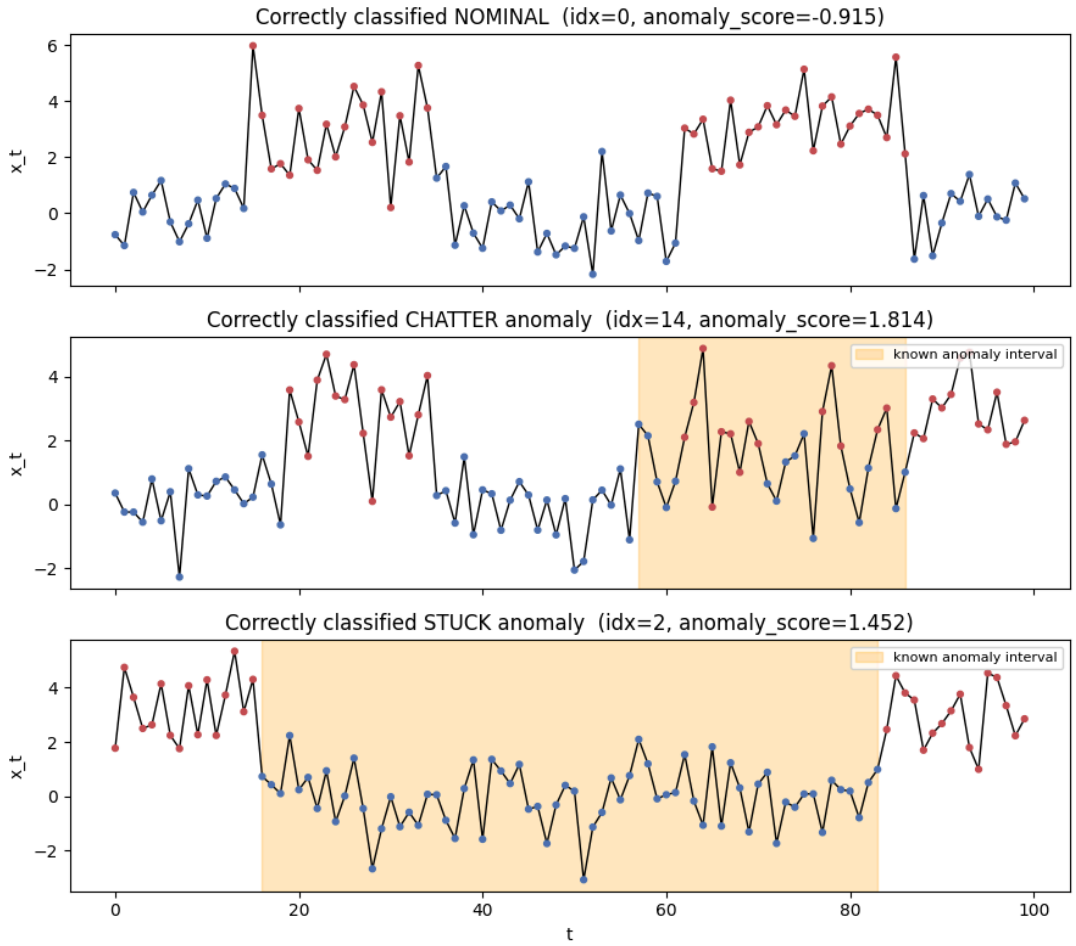


Figure 10: Viterbi-decoded hidden paths for a correctly-classified nominal, chatter, and stuck sequence. Blue = Low, red = High; orange band = known anomaly interval.

r_{test}	0.01	0.05	0.10	0.20
OC-SVM AUC	0.8049	0.7804	0.7444	0.7737
HMAD AUC	0.9860	0.9957	1.0000	0.9981

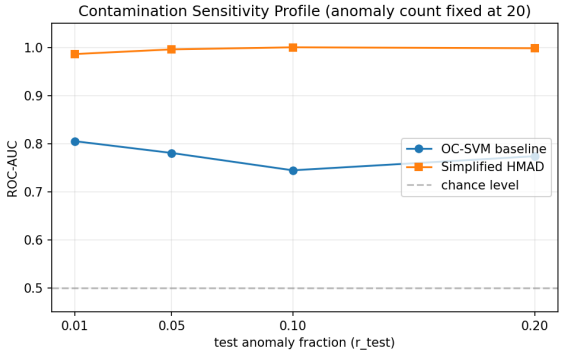


Figure 13: ROC-AUC vs. test contamination fraction for both methods.

HMAD stays above 0.98 across the whole range, while OC-SVM stays roughly in the mid/high 0.7s. The gap never closes.

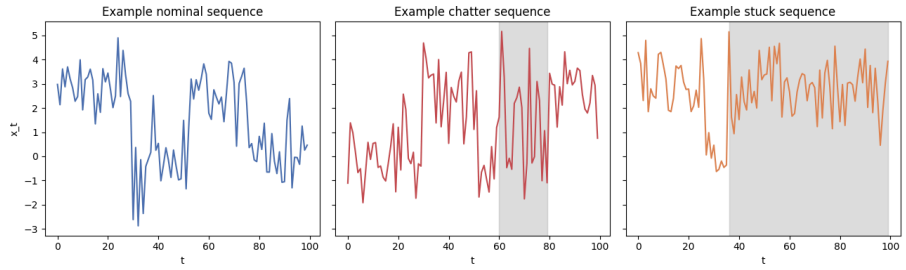


Figure 11: Example nominal, chatter, and stuck sequences from the $r_{\text{test}} = 0.1$ test set (gray band = known anomaly interval).

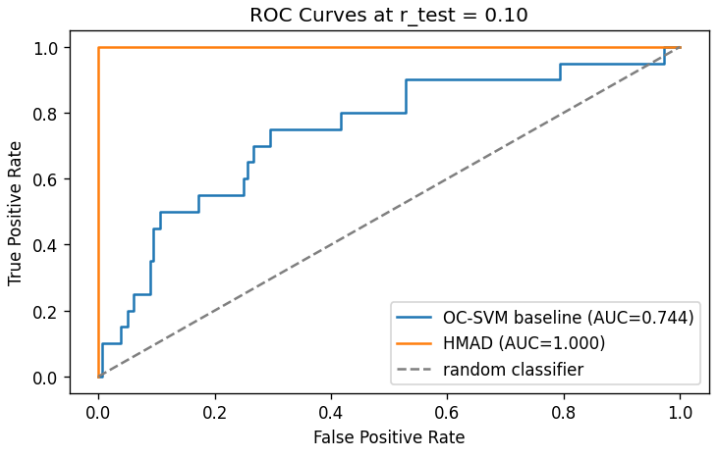


Figure 12: ROC curves at $r_{\text{test}} = 0.1$: HMAD dominates the OC-SVM baseline.

8 Conclusion

The central question of the project was whether a sequence-aware detector really deserves its extra complexity, compared with a fixed-vector baseline. On a dataset where the anomaly lives only in switching dynamics — never in the raw values — the answer here is a clear and measured yes. Global summaries gave only partial separation; stuck retained some indirect signal (Section 1). OC-SVM, no matter which fixed-vector representation we gave it, stayed around AUC 0.72–0.80. HMAD, trained end-to-end with no oracle knowledge of the true generative process, reached AUC ~ 0.999 and recovered emission means and a transition matrix that are almost the same as the ones that generated the data. If we remove the one ingredient that makes HMAD sequence-aware — Viterbi decoding — its performance falls straight to chance.

The more general lesson is not only about this specific algorithm. It is about how we should think about anomaly problems in general. Before we use a fixed-vector detector, it is useful to ask if the anomaly we care about could be a *structural* one, hidden in timing or ordering rather than in magnitude. If this can happen, no amount of clever feature engineering on single values will find it — we need a model that looks at the sequence as a sequence.